

GUÍA PRÁCTICA DE LABORATORIO

Pruebas de una API REST con la extensión REST Client de Visual Studio Code

Datos generales	
Curso	JavaScript Avanzado
Tema	Pruebas manuales de APIs REST con archivos .http
Modalidad	Laboratorio (práctica guiada en clase)
Duración estimada	90 minutos
Docente	Ing. David Reyna Barreto
Estudiante	
Sección / Fecha	

1. Logro de aprendizaje

Al finalizar la práctica, **verificas el funcionamiento de una API REST** escribiendo y ejecutando peticiones HTTP en archivos **.http** con la extensión **REST Client** de Visual Studio Code. Usarás variables, peticiones encadenadas y entornos, y **compararás** cada respuesta con el código de estado esperado para detectar errores de la API.

Trabajarás sobre la API de usuarios construida en la guía anterior (**API REST con Node.js 20, Express 5 y MySQL**). Si aún no la tienes funcionando, complétala antes de empezar.

2. ¿Qué es REST Client?

REST Client es una extensión gratuita de Visual Studio Code que permite enviar peticiones HTTP escritas como **texto plano** dentro de un archivo con extensión **.http** o **.rest**, y ver la respuesta en un panel al lado del código (Mao, s. f.). A diferencia de Postman, las pruebas no quedan guardadas en una cuenta ni en una aplicación aparte: forman parte del proyecto, se versionan con Git y cualquier compañero del equipo puede ejecutarlas al clonar el repositorio.

Aspecto	REST Client	Postman
Dónde se ejecuta	Dentro de VS Code	Aplicación independiente
Formato de las pruebas	Archivo de texto .http	Colecciones en JSON exportable
Versionado con Git	Directo: el archivo vive en el repositorio	Requiere exportar e importar la colección
Requiere cuenta	No	Sí, para sincronizar
Variables y entornos	Sí	Sí
Aserciones automáticas (tests)	No: la verificación es visual	Sí, con scripts

 **Nota:** REST Client sirve para **pruebas manuales y exploratorias**. Para pruebas que se ejecutan solas y fallan si algo cambia, usa las pruebas automatizadas con **node --test** de la guía anterior. Ambas se complementan.

3. Requisitos previos

- Visual Studio Code actualizado.
- La API de usuarios de la guía anterior, con su base de datos `crud_db` creada.
- MySQL en ejecución y el archivo `.env` configurado.

4. Desarrollo de la práctica

Paso 1. Instalar la extensión

1. En VS Code abre el panel de extensiones con `Ctrl+Shift+X`.
2. Busca **REST Client**. Verifica que el autor sea **Huachao Mao** (identificador `humao.rest-client`); hay extensiones con nombres parecidos.
3. Haz clic en **Install**. No hace falta reiniciar el editor.

✓ **Punto de control:** En la lista de extensiones instaladas aparece **REST Client** de Huachao Mao.

Paso 2. Levantar la API

Abre la carpeta del proyecto en VS Code, inicia MySQL y, en la terminal integrada, ejecuta:

```
npm run dev
```

Debes ver **Conectado a MySQL** y **Servidor corriendo en http://localhost:3000**. Deja esta terminal abierta durante toda la práctica.

Paso 3. Crear el archivo de pruebas y enviar la primera petición

En la raíz del proyecto crea la carpeta `http` y dentro el archivo `usuarios.http`. Escribe solo esta línea:

```
http/usuarios.http
GET http://localhost:3000/api/users
```

Sobre la línea aparecerá un enlace pequeño **Send Request**. Haz clic en él (o coloca el cursor en la línea y pulsa `Ctrl+Alt+R`). A la derecha se abre el panel de respuesta, que tiene tres partes:

Parte	Ejemplo	Qué te dice
Línea de estado	HTTP/1.1 200 OK	El código de estado: lo primero que debes revisar
Cabeceras	Content-Type: application/json; charset=utf-8	Metadatos: tipo de contenido, tamaño, CORS
Cuerpo	[{"id": 1, "name": "Ana", ...}]	Los datos que devolvió la API

💡 **Nota:** Si el enlace **Send Request** no aparece, revisa que el archivo termine en `.http` y que en la esquina inferior derecha de VS Code el modo de lenguaje diga **HTTP**.

✓ **Punto de control:** El panel de respuesta muestra **HTTP/1.1 200 OK** y un arreglo JSON (puede estar vacío: `[]`).

Paso 4. Entender la anatomía de una petición

Cada petición tiene siempre el mismo orden. El detalle más importante es la **línea en blanco** entre las cabeceras y el cuerpo: sin ella, REST Client interpreta el JSON como si fueran cabeceras y la API no recibe los datos.

```

### Crear un usuario                                <- separador y título (opcional)
POST http://localhost:3000/api/users                <- método y URL
Content-Type: application/json                       <- cabeceras (una por línea)
                                                    <- LÍNEA EN BLANCO obligatoria
{                                                    <- cuerpo
  "name": "Ana Torres",
  "email": "ana@mail.com"
}
  
```

Elemento	Sintaxis	Para qué sirve
Separador	###	Divide el archivo en varias peticiones. El texto que sigue es un título
Comentario	# texto o // texto	Notas que no se envían
Variable de archivo	@nombre = valor	Define un valor reutilizable en todo el archivo
Uso de variable	{{nombre}}	Inserta el valor de la variable
Petición con nombre	# @name crearUsuario	Guarda la respuesta para usarla en otras peticiones
Variable dinámica	{{\${randomInt 1000 99999}}}	Genera un valor distinto en cada envío

Paso 5. Usar variables de archivo

Escribir la URL completa en cada petición es repetitivo y propenso a errores. Reemplaza el contenido del archivo por el siguiente. Si mañana la API cambia de puerto, solo modificas una línea:

```

http/usuarios.http
@baseUrl = http://localhost:3000/api
@contentType = application/json

### 1. Listar usuarios -> 200
GET {{baseUrl}}/users
  
```

Paso 6. Crear, consultar y actualizar encadenando peticiones

Cuando creas un usuario, la base de datos le asigna un **id** que no conoces de antemano. En lugar de copiarlo a mano, **ponle nombre a la petición** con **# @name** y lee su respuesta desde las siguientes con la sintaxis **{{nombre.response.body.\$.campo}}**, donde **\$.campo** es una ruta JSONPath (Mao, s. f.). Agrega al archivo:

```

### 2. Crear un usuario -> 201
# @name crearUsuario
POST {{baseUrl}}/users
Content-Type: {{contentType}}

{
  "name": "Ana Torres",
  "email": "ana{{$randomInt 1000 99999}}@mail.com"
}

### 3. Obtener el usuario recién creado -> 200
GET {{baseUrl}}/users/{{crearUsuario.response.body.$id}}

### 4. Actualizar el usuario creado -> 200
PUT {{baseUrl}}/users/{{crearUsuario.response.body.$id}}
Content-Type: {{contentType}}

{
  "name": "Ana María Torres"
}

```

Observa el email: `{{ $randomInt 1000 99999 }}` genera un número aleatorio en cada envío. Así puedes repetir la petición 2 las veces que quieras sin chocar con la restricción **UNIQUE** de la base de datos.

⚠ Importante: Las peticiones encadenadas dependen del orden. Si envías la 3 sin haber enviado antes la 2, REST Client no puede resolver `{{crearUsuario.response.body.$id}}` y la URL queda incompleta. Ejecuta siempre la petición con nombre primero.

✓ Punto de control: La petición 2 devuelve **201** con el **id** asignado; la 3 devuelve ese mismo usuario; la 4 devuelve el nombre actualizado a "Ana María Torres".

Paso 7. Probar los casos de error

Una API no se prueba solo por el "camino feliz". La mitad del trabajo es comprobar que **rechaza correctamente** lo que no debe aceptar. Agrega estas peticiones al archivo:

```

### 5. Asignación masiva: enviar id y role -> 201 (sin role, id asignado por la BD)
# @name crearConRol
POST {{baseUrl}}/users
Content-Type: {{contentType}}

{
  "name": "Luis Pérez",
  "email": "luis{{$randomInt 1000 99999}}@mail.com",
  "id": 999,
  "role": "admin"
}

### 6. Email duplicado -> 409
POST {{baseUrl}}/users
Content-Type: {{contentType}}

{
  "name": "Ana Duplicada",
  "email": "{{crearUsuario.response.body.$email}}"
}

### 7. Campos obligatorios vacíos -> 400
POST {{baseUrl}}/users
Content-Type: {{contentType}}

{}

```

```

### 8. Email con formato inválido -> 400
POST {{baseUrl}}/users
Content-Type: {{contentType}}

```

```

{
  "name": "Pedro",
  "email": "no-es-un-email"
}

```

```

### 9. JSON mal formado -> 400
POST {{baseUrl}}/users
Content-Type: {{contentType}}

```

```

{ "name": "Rosa",

```

En la petición 5, revisa con atención la respuesta: el **id** **no** debe ser 999 y no debe aparecer **role**. Si aparece, la API tiene una vulnerabilidad de asignación masiva. En la petición 6 se reutiliza el email que devolvió la petición 2, así el duplicado está garantizado.

Completa el archivo con las pruebas de id y eliminación:

```

### 10. PUT sin campos permitidos -> 400
PUT {{baseUrl}}/users/{{crearUsuario.response.body.$.id}}
Content-Type: {{contentType}}

```

```

{
  "foo": 1
}

```

```

### 11. Id que no existe -> 404
GET {{baseUrl}}/users/999999

```

```

### 12. Id no numérico -> 400
GET {{baseUrl}}/users/abc

```

```

### 13. Eliminar el usuario creado en la petición 5 -> 204
DELETE {{baseUrl}}/users/{{crearConRol.response.body.$.id}}

```

```

### 14. Eliminarlo otra vez -> 404
DELETE {{baseUrl}}/users/{{crearConRol.response.body.$.id}}

```

```

### 15. Ruta inexistente -> 404
GET {{baseUrl}}/productos

```

✓ **Punto de control:** Tienes 15 peticiones en el archivo. El archivo completo está en el **Anexo A** para que compares el tuyo.

Paso 8. Ejecutar y registrar los resultados

Ejecuta las 15 peticiones **en orden** y completa la columna **Obtenido** con el código que muestra la línea de estado. Si algún código no coincide, anota en **Observación** qué respondió la API: ese es un hallazgo.

#	Prueba	Esperado	Obtenido	Observación
1	Listar usuarios	200		
2	Crear usuario	201		
3	Obtener usuario creado	200		
4	Actualizar usuario	200		

#	Prueba	Esperado	Obtenido	Observación
5	Asignación masiva (id y role)	201, sin role		
6	Email duplicado	409		
7	Cuerpo vacío	400		
8	Email inválido	400		
9	JSON mal formado	400		
10	PUT sin campos permitidos	400		
11	Id inexistente	404		
12	Id no numérico	400		
13	Eliminar usuario	204		
14	Eliminar de nuevo	404		
15	Ruta inexistente	404		

✓ **Punto de control:** Los 15 códigos obtenidos coinciden con los esperados. Toma capturas del panel de respuesta de las pruebas 5, 6, 9 y 13.

Paso 9. Configurar entornos

En un proyecto real la misma API corre en varios lugares: tu computadora, el servidor del laboratorio, producción. Los **entornos** permiten cambiar todas las URLs con un clic. Crea el archivo `.vscode/settings.json` en la raíz del proyecto:

```
.vscode/settings.json
{
  "rest-client.environmentVariables": {
    "$shared": {
      "contentType": "application/json"
    },
    "local": {
      "baseUrl": "http://localhost:3000/api"
    },
    "laboratorio": {
      "baseUrl": "http://localhost:3001/api"
    }
  }
}
```

Las variables de `$shared` están disponibles en todos los entornos. Ahora **borra o comenta** las dos líneas `@baseUrl` y `@contentType` al inicio de `usuarios.http`: las variables de archivo tienen prioridad sobre las de entorno, así que si las dejas, el cambio de entorno no tendrá efecto (Mao, s. f.).

Para elegir el entorno pulsa **Ctrl+Alt+E** o haz clic en el nombre del entorno en la barra inferior derecha de VS Code. Selecciona **local** y vuelve a enviar la petición 1.

✓ **Punto de control:** Con el entorno **local** la petición 1 responde 200. Si cambias a **laboratorio** (puerto 3001, donde no hay API), obtienes un error de conexión: eso demuestra que el entorno está funcionando.

Paso 10. Funciones útiles

Función	Atajo (Windows/Linux)	Uso
Enviar petición	Ctrl+Alt+R	Envía la petición donde está el cursor
Repetir la última	Ctrl+Alt+L	Reenvía la última petición enviada
Cancelar	Ctrl+Alt+K	Detiene una petición que tarda demasiado
Historial	Ctrl+Alt+H	Lista las peticiones enviadas recientemente
Cambiar entorno	Ctrl+Alt+E	Alterna entre local, laboratorio, etc.
Generar código	Ctrl+Alt+C	Convierte la petición en código (JavaScript fetch, Python, etc.)
Copiar como cURL	Clic derecho › Copy Request As cURL	Obtiene el comando curl equivalente para compartirlo

En macOS reemplaza **Ctrl** por **Cmd**. Prueba **Generar código** sobre la petición 2 y elige JavaScript: verás el **fetch()** equivalente, útil cuando construyas el frontend que consumirá esta API.

Paso 11. Versionar las pruebas

```
git add http/usuarios.http .vscode/settings.json
git commit -m "Pruebas de la API con REST Client"
git push
```

 **Seguridad:** Nunca escribas contraseñas ni tokens reales dentro de un archivo **.http** o de **settings.json** que vayas a subir a GitHub. Si una API los requiere, léelos desde tu **.env** con **{{\${dotenv NOMBRE_VARIABLE}}}**.

5. Errores frecuentes y cómo resolverlos

Síntoma	Causa probable	Solución
No aparece "Send Request"	El archivo no termina en .http o la extensión no está instalada	Renombra el archivo y revisa el Paso 1
Connection is being rejected / ECONNREFUSED	La API no está corriendo o el puerto es otro	Ejecuta npm run dev y revisa baseUrl
400 "name es obligatorio" aunque enviaste datos	Falta la línea en blanco antes del cuerpo o la cabecera Content-Type	Revisa la anatomía del Paso 4
La URL contiene {{crearUsuario...}} sin resolver	No se ejecutó antes la petición con nombre	Envía primero la petición 2 (o la 5)
409 al repetir la petición 2	El email ya existe en la base de datos	Usa {{\${randomInt 1000 99999}}} en el email
El cambio de entorno no hace nada	Las variables @baseUrl del archivo tienen prioridad	Comenta las variables de archivo (Paso 9)
Error en settings.json	JSON mal formado (coma final, comillas simples)	VS Code subraya el error en rojo; corrígelo

6. Lista de cotejo

Entregable: el enlace a tu repositorio de GitHub con `http/usuarios.http` y `.vscode/settings.json`, y un PDF con la tabla del Paso 8 completa, las capturas solicitadas y tus respuestas a las preguntas de reflexión. Nombre del PDF: `Apellido_Nombre_RestClient.pdf`.

El docente verificará cada criterio. Antes de entregar, márcalo tú mismo en la columna **Autoevaluación**.

N.º	Criterio	Puntaje	Autoevaluación	Docente
1	El archivo <code>usuarios.http</code> contiene las 15 peticiones separadas con <code>###</code> y con título.	3		
2	Usa variables de archivo o de entorno en lugar de URLs repetidas.	2		
3	Encadena peticiones con <code># @name</code> y <code>{{nombre.response.body.\$.campo}}</code> .	3		
4	Usa variables dinámicas para que las pruebas se puedan repetir sin error 409.	2		
5	La tabla del Paso 8 está completa y los códigos coinciden con los esperados.	4		
6	Las capturas evidencian las pruebas 5, 6, 9 y 13.	2		
7	Configura al menos dos entornos en <code>.vscode/settings.json</code> y los alterna correctamente.	2		
8	Las respuestas a las preguntas de reflexión son correctas y están argumentadas.	2		
	Total	20		

7. Preguntas de reflexión

Responde brevemente, con tus propias palabras, al final de tu PDF de evidencias:

- ¿Qué ventaja tiene guardar las pruebas en un archivo `.http` dentro del repositorio frente a tenerlas en una aplicación externa?
- ¿Por qué la petición 3 usa `{{crearUsuario.response.body.$.id}}` en lugar de un número fijo como `/users/1`?
- En la prueba 5, ¿qué habrías concluido si la respuesta hubiera incluido `"role": "admin"`? ¿Qué capa de la API habría que corregir?
- ¿Qué pasa si quitas la línea en blanco entre `Content-Type` y el cuerpo JSON? Pruébalo y explica el resultado.
- ¿En qué se diferencian estas pruebas manuales de las pruebas automatizadas con `node --test`? ¿Cuándo usarías cada una?

8. Referencias

Mao, H. (s. f.). *REST Client* [Extensión de Visual Studio Code]. GitHub. <https://github.com/Huachao/vscode-restclient>

MDN Web Docs. (s. f.). *Códigos de estado de respuesta HTTP*. Mozilla. <https://developer.mozilla.org/es/docs/Web/HTTP/Status>

Anexo A. Archivo completo usuarios.http

Este es el archivo final de la práctica, antes de configurar los entornos del Paso 9. Úsalo para comparar el tuyo si alguna petición no responde como se espera.

```
http/usuarios.http
# =====
# Pruebas de la API de usuarios con REST Client (VS Code)
# Ejecuta las peticiones EN ORDEN: haz clic en "Send Request"
# o coloca el cursor dentro de la petición y pulsa Ctrl+Alt+R
# =====

@baseUrl = http://localhost:3000/api
@contentType = application/json

### 1. Listar usuarios -> 200
GET {{baseUrl}}/users

### 2. Crear un usuario -> 201
# @name crearUsuario
POST {{baseUrl}}/users
Content-Type: {{contentType}}

{
  "name": "Ana Torres",
  "email": "ana{{$randomInt 1000 99999}}@mail.com"
}

### 3. Obtener el usuario recién creado -> 200
GET {{baseUrl}}/users/{{crearUsuario.response.body.$id}}

### 4. Actualizar el usuario creado -> 200
PUT {{baseUrl}}/users/{{crearUsuario.response.body.$id}}
Content-Type: {{contentType}}

{
  "name": "Ana María Torres"
}

### 5. Asignación masiva: enviar id y role -> 201 (sin role, id asignado por la BD)
# @name crearConRol
POST {{baseUrl}}/users
Content-Type: {{contentType}}

{
  "name": "Luis Pérez",
  "email": "luis{{$randomInt 1000 99999}}@mail.com",
  "id": 999,
  "role": "admin"
}

### 6. Email duplicado -> 409
POST {{baseUrl}}/users
Content-Type: {{contentType}}

{
  "name": "Ana Duplicada",
  "email": "{{crearUsuario.response.body.$email}}"
}

### 7. Campos obligatorios vacíos -> 400
POST {{baseUrl}}/users
Content-Type: {{contentType}}
```

```
{}
```

8. Email con formato inválido -> 400
POST {{baseUrl}}/users
Content-Type: {{contentType}}

```
{
  "name": "Pedro",
  "email": "no-es-un-email"
}
```

9. JSON mal formado -> 400
POST {{baseUrl}}/users
Content-Type: {{contentType}}

```
{ "name": "Rosa",
```

10. PUT sin campos permitidos -> 400
PUT {{baseUrl}}/users/{{crearUsuario.response.body.\$id}}
Content-Type: {{contentType}}

```
{
  "foo": 1
}
```

11. Id que no existe -> 404
GET {{baseUrl}}/users/999999

12. Id no numérico -> 400
GET {{baseUrl}}/users/abc

13. Eliminar el usuario creado en la petición 5 -> 204
DELETE {{baseUrl}}/users/{{crearConRol.response.body.\$id}}

14. Eliminarlo otra vez -> 404
DELETE {{baseUrl}}/users/{{crearConRol.response.body.\$id}}

15. Ruta inexistente -> 404
GET {{baseUrl}}/productos